UNIVERSITÀ DELLA CALABRIA

DIPARTIMENTO DI INGEGNERIA INFORMATICA, MODELLISTICA, ELETTRONICA E SISTEMISTICA

CORSO DI LAUREA TRIENNALE IN INGEGNERIA INFORMATICA

ELETTRONICA DIGITALE

RELAZIONE PRIMO PROGETTINO – SOMMATORE A 16 BIT

Petrassi Maria Jose' – 251439

Ponte Andrea – 251050

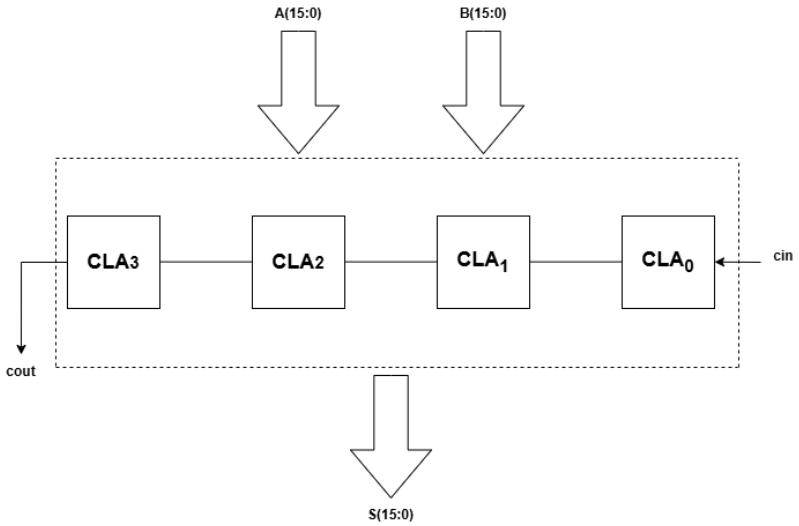
Ruffolo Emilio – 250521

Detti: "i tri 'da chiazza"

Traccia Progetto

Progettare e realizzare un sommatore a 16 bit per numeri unsigned, composto da quattro moduli Carry Look-Ahead Adder (CLA) a 4 bit collegati tra loro secondo la struttura ripple carry. Scrivere il test bench per la verifica funzionale del circuito, includendo almeno sette combinazioni di test significative.

Redigere infine una relazione tecnica che descriva l'architettura, il funzionamento, i risultati della simulazione e le conclusioni.



Introduzione al VHDL

Il circuito è stato descritto in linguaggio VHDL, simulato tramite test bench e verificato con diverse combinazioni di input significative. La relazione illustra l'architettura del circuito, il funzionamento dei moduli, i risultati della simulazione e le conclusioni tratte.

VHDL (acronimo di Very High Speed Integrated Circuit Hardware Description Language) è un linguaggio di descrizione hardware utilizzato per progettare circuiti digitali. Permette di modellare il comportamento e la struttura di sistemi elettronici, come porte logiche, registri, contatori, e interi processori.

Con VHDL si possono definire:

- **Entità**: la parte esterna del modulo, con ingressi e uscite;
- **Architetture**: la parte interna, che descrive come il modulo funziona.

VHDL supporta la descrizione:

- **Comportamentale** (come si comportano i segnali);
- **Strutturale** (come sono collegati i moduli);
- **Dataflow** (tramite equazioni logiche).

Il VHDL è diventato lo standard IEEE dell'industria per la simulazione, la sintesi e la verifica dei circuiti digitali.

Architettura del sommatore a 16 bit

Per comprendere l'architettura del sommatore realizzato, è utile partire dal concetto di Ripple Carry Adder (RCA). Questo tipo di sommatore è composto da una serie di moduli elementari (come sommatrici a 1 bit o a 4 bit), ciascuno dei quali calcola la somma di una porzione dei bit e trasmette il carry al modulo successivo. Il nome "ripple" deriva proprio dal fatto che il carry si "propaga" da un modulo all'altro, come un'onda, rallentando il tempo totale di calcolo. Nel sommatore ripple tradizionale, ogni modulo deve attendere il calcolo del carry dal modulo precedente prima di poter completare la propria somma. Questo meccanismo, sebbene semplice, introduce un ritardo crescente con l'aumentare del numero di bit, poiché il carry deve passare da un modulo all'altro. Infatti, in un'ALU moderna il calcolo del singolo bit di somma e di resto forniti dal primo blocco è di circa $5\ ns$, quindi un'operazione di somma per un processore a $64\ bit$ impiegherebbe circa $32\ ns$, portando ad una frequenza di clock massima di $0{,}03\ GHz$.

Per superare questo limite, si utilizza il Carry Look-Ahead Adder (CLA), una tecnica che permette di calcolare i carry in modo anticipato, senza dover attendere la propagazione sequenziale. Il CLA si basa su due segnali fondamentali:

- **Propagate** (p): indica se il modulo trasmette il carry in ingresso.
- **Generate** (g): indica se il modulo genera un carry indipendentemente dal carry in ingresso.

Grazie a questi due nuovi segnali, è possibile calcolare direttamente tutti i carry interni in parallelo con delle espressioni logiche, evitando la dipendenza sequenziale. I bit p e g sono ricavati come segue:

$$\boldsymbol{p_i = a_i \oplus b_i} \qquad \boldsymbol{g_i = a_i \cdot b_i}$$

Permettendoci di calcolare il bit di somma S e di riporto c dell'i-esimo blocco come:

$$\boldsymbol{S_i = p_i \oplus c_i} \qquad \boldsymbol{c_{i+1} = p_i \cdot c_i + g_i}$$

Come richiesto dalla consegna, andremo a costruire un sommatore a 16 bit seguendo la struttura di un Ripple Carry Adder dove ogni modulo che lo compone non è un semplice Full Adder, ma bensì un Carry Look Ahead Adder da 4 bit. Dunque, in totale il circuito sarà composto da quattro moduli CLA, ciascuno responsabile della somma di 4 bit dei due operandi A e B.

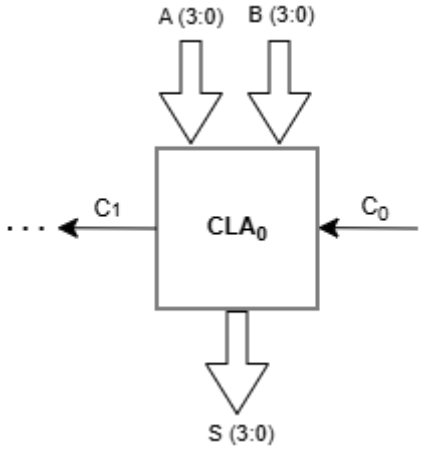
In particolare, l'i-esimo modulo riceve in ingresso:

- Gli i-esimi 4 bit di A;
- Gli i-esimi 4 bit di B;
- Il bit di carry C_i

Producendo in uscita:

- Gli i-esimi 4 bit della somma parziale S
- Il bit di carry C_{i+1}

Tramite questa implementazione il tempo di propagazione del carry non dipende più da tutti i 16 bit del sommatore, ma solo dai 4 bit gestiti da ciascun modulo. Mentre in un ripple carry tradizionale, il carry deve attraversare tutti i bit, generando un ritardo proporzionale alla lunghezza dell'operando, in questa soluzione il ritardo interno di ciascun modulo CLA è fisso e limitato a 4 bit, poiché il carry viene calcolato in modo parallelo all'interno del modulo. Il ritardo complessivo del circuito è quindi determinato dalla propagazione del carry tra i 4 moduli, e non più tra tutti i 16 bit.



La struttura è composta da tre file principali:

1. **cla4bit.vhd**: contiene la descrizione del modulo CLA a 4 bit. Questo modulo calcola la somma di 4 bit e il carry in uscita, utilizzando la logica del propagate e generate per anticipare il calcolo dei carry interni;
2. **sum16bit.vhd**: rappresenta il sommatore completo a 16 bit. Qui vengono istanziati quattro moduli cla4bit, collegati tra loro secondo la struttura del ripple carry. I segnali A e B vengono suddivisi in blocchi da 4 bit, e i carry vengono propagati da un modulo all'altro tramite una catena di segnali C;
3. **test.vhd**: è il test-bench utilizzato per la verifica funzionale del circuito. In questo file vengono generate diverse combinazioni di input per A e B, e viene osservato il comportamento del circuito in risposta a ciascun caso. Il carry iniziale è impostato a '0', e il risultato della somma viene monitorato tramite il segnale S.

1. Implementazione del Carry Look Ahead a 4 bit in VIVADO

In VHDL, la sezione entity definisce l'interfaccia esterna di un modulo, cioè specifica quali segnali entrano ed escono dal componente. È come una "dichiarazione pubblica" che descrive cosa il modulo riceve e cosa restituisce, senza entrare nel dettaglio di come funziona internamente.

```
entity cla4bit is
    port(A,B: in bit_vector(3 downto 0);
         cin: in bit;
         cout: out bit;
         S: out bit_vector(3 downto 0));
end cla4bit;
```

Notiamo come in questo caso gli input ricevuti sono:

- $\boldsymbol{A}, \boldsymbol{B}$: 2 vettori da 4 bit che rappresentano gli operandi;
- $\boldsymbol{Cin}$: un bit di carry che rappresenta il resto del modulo precedente.

E le uscite prodotte:

- $\boldsymbol{S}$**:** un vettore da 4 bit che rappresenta la somma;
- $\boldsymbol{Cout}$**:** un bit di carry che rappresenta il resto prodotto.

La sezione architecture rappresenta la descrizione interna di un modulo, ovvero come esso si comporta e come è strutturato. Esistono tre principali modalità di descrizione:

- Descrizione **Data Flux**: il comportamento del modulo viene espresso tramite equazioni booleane, evidenziando la logica combinatoria.
- Descrizione **Behavioural**: si utilizza un insieme di istruzioni sequenziali per modellare il comportamento del modulo nel tempo.
- Descrizione **Strutturale**: si definisce il modulo attraverso l'interconnessione di componenti, evidenziando la sua composizione gerarchica.

```
architecture my_cla4bit of cla4bit is
    signal p,g: bit_vector(3 downto 0);
    signal C: bit_vector(4 downto 0);

    begin
        p <= A xor B;
        g <= A and B;

        C(0) <= cin;

        C(1) <= g(0) or (p(0) and C(0));

        C(2) <= g(1) or (p(1) and g(0)) or (p(1) and p(0) and C(0));

        C(3) <= g(2) or (p(2) and g(1)) or (p(2) and p(1) and g(0))
                or (p(2) and p(1) and p(0) and C(0));

        C(4) <= g(3) or (p(3) and g(2)) or (p(3) and p(2) and g(1))
                or (p(3) and p(2) and p(1) and g(0)) or (p(3) and p(2) and p(1)
                                                         and p(0) and C(0));
        cout <= C(4);
        S <= p xor C(3 downto 0);

end my_cla4bit;
```

Nel caso del cla4bit l'architecture ha il nome di my_cla4bit ed implementa la logica del Carry Look Ahead adder a 4 bit.

All'interno dell'architettura la prima cosa che notiamo è la dichiarazione di alcuni segnali locali tramite l'uso della parola chiave signal. Questi sono visibili solo all'interno dell'architettura stessa e possono servire per collegare componenti, memorizzare stati intermedi o gestire la logica interna del modulo.

Nel nostro caso è necessario definire i segnali di Propagate e Generate (necessari per la logica del CLA) definiti come vettore di bit, ed i bit di Carry interni del singolo CLA, anch'essi definiti come vettore di bit, dove $C(0)$ è il Cin e $C(4)$ è il $Cout$.

Subito dopo segue la sezione begin, dove si trovano tutte le istruzioni che definiscono il comportamento del modulo. Queste istruzioni vengono eseguite simultaneamente, come avviene nei circuiti reali. Le prime due istruzioni sono assegnazioni dirette ai segnali p e g (calcolati come specificato in precedenza $p_i = a_i \oplus b_i \quad g_i = a_i \cdot b_i$), seguono poi i calcoli di tutti i bit di carry e di somma (usando $S_i = p_i \oplus c_i \qquad c_{i+1} = p_i \cdot c_i + g_i$). In questo caso siamo obbligati a calcolare i bit di carry uno ad uno (e non possiamo usare operatori bitwise come abbiamo fatto per p, g ed s), in quanto le espressioni che determinano ciascun bit non seguono una struttura uniforme o ricorsiva, ma variano in complessità man mano che si procede.

2. Implementazione del Sommatore a 16 bit

Nel modulo sum16bit, la sezione entity definisce l'interfaccia esterna del sommatore a 16 bit. Essa è strutturata in maniera analoga a quella del ripple carry adder a 4 bit studiato in classe.

```
entity sum16bit is
    port(A,B: in bit_vector(15 downto 0);
         cin: in bit;
         cout: out bit;
         S: out bit_vector(15 downto 0));
end sum16bit;
```

Gli ingressi ricevuti sono:

- $\boldsymbol{A, B}$: due vettori da 16 bit che appresentano gli operandi da sommare;
- $\boldsymbol{cin}$: un bit di carry iniziale che viene solitamente impostato a '0'.

Mentre in uscita il modulo produce:

- $\boldsymbol{S}$: un vettore da 16 bit che rappresenta il risultato della somma;
- $\boldsymbol{cout}$: il bit di carry finale.

```
8   architecture my_sum16bit of sum16bit is
9       signal C: bit_vector(4 downto 0);
10      component cla4bit
11          port(A, B: in bit_vector(3 downto 0);
12               cin: in bit;
13               cout: out bit;
14               S: out bit_vector(3 downto 0));
15      end component;
16      begin
17
18          C(0) <= cin;
19          cout <= C(4);
20          myFor: for i in 0 to 3 generate
21              CLAi: cla4bit port map(A(i*4+3 downto i*4), B(i*4+3 downto i*4), C(i), C(i+1),S(i*4+3 downto i*4));
22          end generate;
23  end my_sum16bit;
```

La sezione architecture del modulo sum16bit, chiamata my_sum16bit, descrive il comportamento interno del sommatore a 16 bit. Con signal dichiariamo il segnale locale C come vettore di 5 bit rappresentante i carry intermedi tra i 4 blocchi CLA. $C(0)$ rappresenta il carry iniziale (cin), mentre $C(4)$ il carry finale ($cout$).

Segue la dichiarazione del componente esterno cla4bit, questa dichiarazione renderà possibile l'istanziazione di più moduli Carry Look Ahead all'interno del mio programma. Come possiamo vedere dal contenuto della port, ogni istanza riceverà due operandi a 4 bit (A e B), un bit di carry in input, e produrrà una somma da 4 bit (S) ed un carry di output.

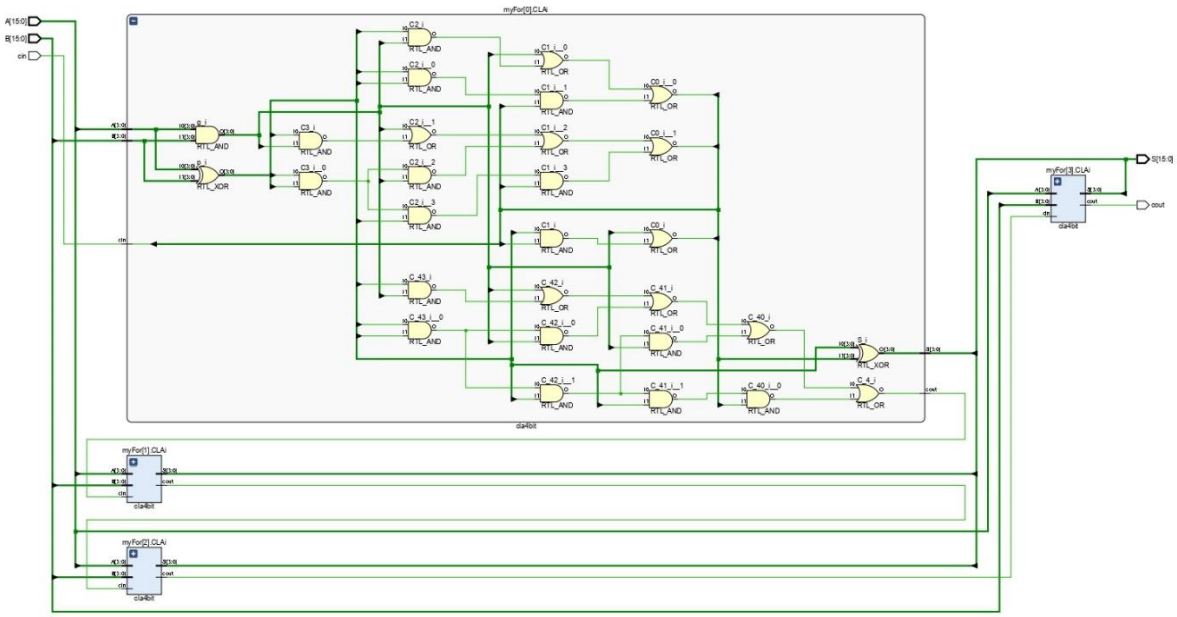
Nella sezione begin vengono definite le istruzioni che caratterizzano il comportamento del modulo. Inizializziamo il primo bit del vettore C con il valore di cin esterno, questo sarà inviato come ingresso per il primo blocco CLA. Successivamente assegniamo a cout il valore all'interno di $C(4)$, questa operazione è valida anche se il valore di $C(4)$ viene calcolato nelle righe successive perché tutto il codice eseguito nella sezione begin è concorrente, cioè, viene eseguito in parallelo, non ha dunque importanza l'ordine in cui lo scriviamo.

Subito dopo viene avviato un ciclo for-generate che si ripete per i valori di i da 0 a 3. In ogni iterazione, viene istanziato un componente cla4bit con etichetta $CLAi$, corrispondente a un blocco da 4 bit del sommatore totale. Nello specifico:

- $A(i*4+3\ downto\ i*4)$: seleziona 4 bit consecutivi dal vettore A, corrispondenti al blocco corrente. Ad esempio, per i=0 prende $A(3\ downto\ 0)$, per $i=1$ prende $A(7\ downto\ 4)$, e così via;
- $B(i*4+3\ downto\ i*4)$: seleziona i corrispondenti 4 bit dal vettore B;
- $C(i)$: fornisce il carry in input per il blocco corrente;
- $C(i+1)$: riceve il carry out generato dal blocco corrente, che sarà usato come carry in per il blocco successivo
- $S(i*4+3\ downto\ i*4)$: raccoglie la somma parziale calcolata dal blocco corrente e la inserisce nella posizione corretta del vettore S.

Questi segnali vengono collegati al componente cla4bit tramite la port map.

A questo punto Vivado ci fornisce la possibilità di generare una rappresentazione grafica del circuito descritto:



3. Simulazione tramite test bench

Il test bench è un modulo VHDL scritto appositamente per simulare e testare un altro. Non ha porte esterne serve esclusivamente per generare segnali di ingresso, osservare le uscite e verificare che il comportamento del modulo sia corretto.

```
library IEEE;
use IEEE.numeric_bit.all;

entity Test is
end Test;

architecture sim_sum16bit of Test is
    component sum16bit
        port(A: in bit_vector(15 downto 0);
             B: in bit_vector(15 downto 0);
             cin: in bit;
             cout: out bit;
             S: out bit_vector(15 downto 0));
    end component;
    signal A,B,S: bit_vector(15 downto 0);
    signal cin, cout: bit;

    begin
        cin <= '0';
        process begin
            for ia in 0 to 15 loop
                for ib in 0 to 15 loop
                    A <= bit_vector(TO_UNSIGNED(ia,16));
                    B <= bit_vector(TO_UNSIGNED(ib,16));
                    wait for 10 ns;
                end loop;
            end loop;
        end process;
        CUT: sum16bit port map(A, B, cin, cout, S);
end sim_sum16bit;
```

Abbiamo importato dalla libreria standard $IEEE$ il pacchetto *numeric_bit* che mette a disposizione funzioni come $TO_UNISIGNED$ che ci verrà in aiuto in seguito per convertire numeri interi in bit vector unsigned.

Essendo in un test, lasciamo la entity vuota vista l'assenza di porte esterne. All'interno dell'architettura, chiamata sim_sum16bit dichiariamo innanzitutto la component sum_16bit che è il modulo che vogliamo testare, e nella port riportiamo i segnali definiti nella sua entity.

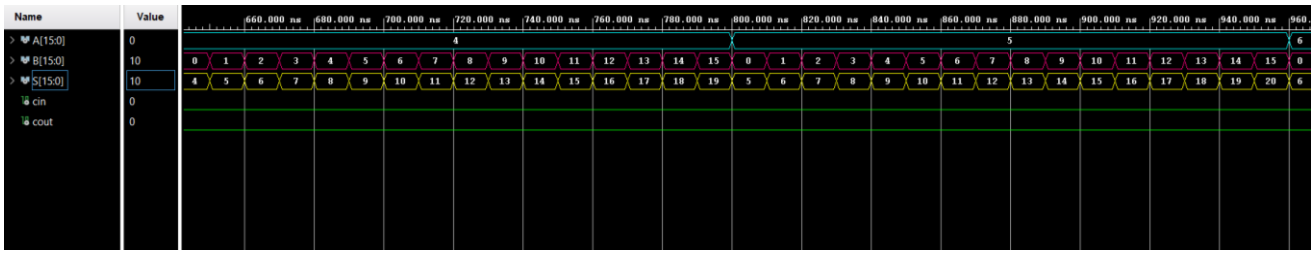
Dichiariamo i segnali locali A, B, S, cin e $cout$, che verranno collegati al modulo sum_16bit simulando ingressi e uscite, e, nella begin, inizializziamo il segnale cin a 0, costante per tutta la simulazione, che indica che il carry iniziale è assente. La sezione process contiene al suo interno istruzioni che verranno eseguite sequenzialmente, e non più in maniera concorrente, come in un qualsiasi linguaggio di programmazione. Questa è fondamentale per la generazione di segnali d'ingresso per il modulo da testare. La generazione di questi avviene tramite l'uso di due for-loop innestati: quello esterno (ia) genera i valori per il bit vector A, mentre quello interno (ib) genera i valori per B. Entrambi variano da 0 a 15.

Procedendo in questo modo, per ogni valore di A vengono testati tutti i valori di B, ottenendo tutte le combinazioni possibili tra i due nell'intervallo da 0 a 15. Assolutamente non tutti i valori assumibili in 16 bit, ma abbastanza per individuare un malfunzionamento nel modulo. Dopo ogni inserimento attendiamo $10\ ns$ prima di testare il prossimo.

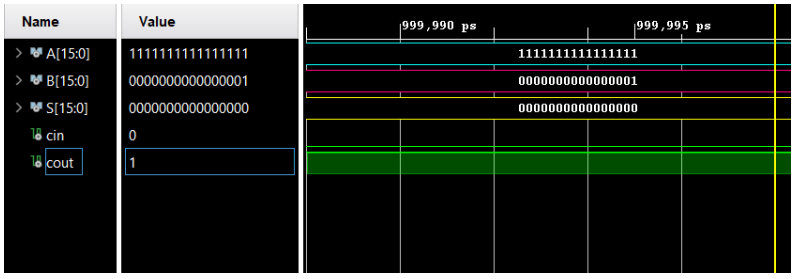
Dopo aver terminato i loop e la process colleghiamo tramite la port map i segnali A, B, cin, cout ed S, all'istanza del componente sum_16bit. Utilizziamo l'etichetta CUT (Circuit Under Test) per convenzione per identificare il modulo di cui si sta eseguendo il test.

La simulazione ci mostra la variazione dei segnali di ingresso A e B nel tempo. Ogni combinazione rimane stabile per $10\ ns$, che è il ritardo che abbiamo impostato. Il segnale A varia più lentamente rispetto a B poiché il ciclo esterno incrementa dopo che il ciclo interno ha completato tutte le iterazioni. Abbiamo ritagliato un tratto ristretto del risultato per evidenziare la sequenza ordinata dei valori generati. In questa porzione vediamo che il segnale A assume 4 e 5, e, per ognuno B assume tutti i valori da 0 a 15. È facile notare che la somma totale riportata in S è corretta.

La simulazione dimostra che il test bench sta stimolando il circuito in modo sistematico e che il modulo riceve ingressi coerenti con quanto progettato. L'uscita S e il carry $cout$, riflettono correttamente la somma dei due operandi (il bit di cout si attiva solo in caso di overflow), confermando la correttezza del comportamento del sommatore.



Proviamo un altro caso di test dove A vale 65535 (ovvero 111111111111111) e B vale 1 (000000000000001). Questo dovrebbe produrre un risultato che non rientra nei 16 bit, facendo alzare il bit di cout. Tuttavia, essendo il sommatore progettato per numeri unsigned, è possibile utilizzare il bit di cout direttamente come diciassettesimo bit.



In conclusione, il sommatore a 16 bit basato su moduli CLA a 4 bit si è dimostrato corretto e più efficiente rispetto al ripple carry puro, con riduzione del ritardo complessivo. La simulazione ha confermato la validità del progetto.